

Solution Requirements

Date	27/06/2026
Team ID	xxxx
Project Name	Personalized Networking Assistant
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements

Functional Requirements (FR) define specific behaviours, functions, or features of the Personalized Networking Assistant. Non-Functional Requirements (NFR) specify the criteria that judge how well the system performs.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	No login/authentication required — the application runs locally as a single-user personal assistant tool for the current release.	Low
2	Authorization levels	Single-user model; no differentiated roles or permission levels are needed at this stage.	Low
3	External interfaces	Integrates with the Google Gemini API (model gemini-2.0-flash, via the google-genai package) for theme extraction and starter generation, and the public Wikipedia API for fact-checking generated claims.	High
4	Transactions processing	Core "transaction" is a request/response cycle between the Streamlit frontend and the FastAPI backend: submit event details -> receive generated starters -> optionally fact-check -> save to history.	Medium
5	Reporting	The system maintains a viewable history log (history.json) of all previously generated conversation starters and a feedback log (feedback.json) capturing user ratings on generated content.	Medium
6	Business rules, etc.	If the Gemini free-tier quota is exhausted or the API call fails, the system automatically falls back to curated starter templates defined in topic_generator.py so the app remains usable.	High
7	Compliance to laws or regulations	No sensitive personal or financial data is collected; only voluntarily entered event/person context is processed. Data is stored locally and is not shared with any third party beyond the Gemini and Wikipedia API calls needed to serve the request.	Medium
8	Other	Feedback capture — users can rate/give feedback on generated starters, stored in feedback.json for future model/prompt improvement.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric
1	Performance & Speed	Conversation starters and theme analysis are generated well within a normal Gemini API round trip.	Response within < 5 seconds per request
2	Scalability	Currently single-user/local; the stateless FastAPI backend is structured so it can later run multiple workers behind a load balancer.	Designed to extend to concurrent multi-user use in a future phase
3	Security & Data Privacy	The Gemini API key is kept in a local .env file and is never committed to the repository (placeholder used on GitHub).	No secrets present in version control
4	Reliability & Availability	Fallback starter templates keep the core app usable even if the Gemini API is unreachable or its quota is exhausted.	App remains functional (core UI) even during external API outages
5	Usability & Accessibility	Clean, glassmorphism-style Streamlit UI with clear, rounded action buttons and stat cards for an intuitive flow.	Users can complete the full flow (input -> starters -> fact-check -> history) without instructions
6	Other (Maintainability)	Backend logic is split into independent, testable modules (event_analyzer.py, topic_generator.py, fact_checker.py).	15 pytest test cases passing, covering core service logic